

👉 Cursor AI MASTER PROMPT (paste all of this into Cursor)

You are acting as a **principal engineer + build system**. Generate a production-grade, mid-complexity demo platform called **PulseCare Telehealth Scheduler** that I can run locally with Docker and deploy to **Google Cloud (GCR + GKE)**, showing **horizontal scalability** with a simple **Kubernetes HPA**. Optimize for fast developer onboarding and a polished demo.

0) Goals & Constraints

- **Use cases (healthcare):**
 - Patients can **book, reschedule, cancel** appointments with providers.
 - Providers manage **availability** (time slots) and **telehealth sessions**.
 - Generate **secure telehealth room links** using **Jitsi** (public SaaS) for the demo.
 - **Reminders** via email (stub + SendGrid-ready integration).
- **Stack:**
 - Backend: **Java 21, Spring Boot 3, Maven, Spring Cloud (Gateway, Config Server, Eureka Service Discovery)**.
 - Microservices: **auth-service, patient-service, provider-service, appointment-service, telehealth-service, notification-service, api-gateway, config-service, discovery-service**.
 - Databases: **PostgreSQL** (transactions: patients/providers/appointments/users), **MongoDB** (audit logs, session metadata).
 - Frontend: **React (Admin/Provider portal) + Vue (Patient portal)**, both as SPAs with sleek, modern UI.
 - Messaging (optional, local only): **Kafka** for domain events (appointment booked/cancelled) to showcase event-driven patterns; keep it **toggled off by default in cloud** to simplify the free demo.

- Observability: **Spring Boot Actuator**, **Micrometer** (Prometheus-ready), **OpenAPI/Swagger** docs per service.
- Security: **Spring Security** JWT with roles (**PATIENT**, **PROVIDER**, **ADMIN**). **BCrypt** for passwords.
- DevX: **docker-compose** for local, **K8s manifests** for GKE, **GitHub Actions** CI/CD to build, test, publish to **GCR** and deploy to **GKE**.
- **HIPAA-friendly posture for demo (not production HIPAA compliance):**
 - TLS-ready ingress.
 - Do **not** log PHI/PII.
 - Mask PII in traces/logs.
 - RBAC per role.
 - Encryption in transit, secrets in K8s (base64) with instructions to switch to a secret manager.

1) Repository & Workspace

Create a monorepo structure:

```
pulsecare-telehealth/
  README.md
  Makefile
  docker-compose.yml
  k8s/
    namespace.yaml
    ingress.yaml
    gateway.yaml
    hpa-appointment.yaml
  postgres/
    statefulset.yaml
    service.yaml
    secret.yaml
  mongo/
```

```
    statefulset.yaml
    service.yaml
    secret.yaml
    <one folder per service>/deployment.yaml + service.yaml +
configmap.yaml + secret.yaml
backend/
  auth-service/
  patient-service/
  provider-service/
  appointment-service/
  telehealth-service/
  notification-service/
  api-gateway/
  config-service/
  discovery-service/
frontend/
  staff-portal-react/
  patient-portal-vue/
scripts/
  init_repo.sh
  local_seed.sh
  gcp_login.sh
  gke_deploy.sh
  gke_destroy.sh
  load_test_hpa.sh
.github/workflows/
  ci-cd.yaml
```

1.1 GitHub remote (impressive name)

- Script `scripts/init_repo.sh` uses **GitHub CLI (gh)** to create a private repo named **pulsecare-telehealth** under my user/org, set main branch, push initial commit, add topics: `spring-boot`, `microservices`, `react`, `vue`, `kubernetes`, `gke`, `healthcare`, `telehealth`.
- If `gh` isn't logged in, print instructions to run `gh auth login`.

- Add a **Contributor Guide** section to README describing how AI (Cursor) assisted and how code was reviewed/edited by me.

2) Backend Services (Spring Boot 3, Maven)

For each service:

- Use Java 21, Maven wrapper, strict Checkstyle/SpotBugs, JUnit 5, Testcontainers (Postgres, Mongo).
- Expose `/actuator/health`, `/actuator/metrics`.
- Publish **OpenAPI** via `springdoc-openapi`.
- Centralized error handling; no PHI in logs.
- Include **sample data loaders** for dev profile.
- **Auth**: `auth-service` issues JWTs (HS256) with short-lived access tokens and roles. Store users in Postgres. Passwords hashed with BCrypt.
- **Config**: `config-service` (Spring Cloud Config, git-backed) stores application configs.
- **Discovery**: `discovery-service` (Eureka).
- **Gateway**: `api-gateway` (Spring Cloud Gateway) with routes to each service, **rate limiting** filter, CORS, correlation IDs.
- **Patient, Provider, Appointment** services use Postgres.
- **Telehealth** service returns signed **Jitsi** room URLs (randomized room slug + optional JWT if you add Jitsi token template), and stores non-PHI session metadata in MongoDB.
- **Notification** service exposes an endpoint to simulate email send; provide SendGrid integration stubs behind a feature flag.
- Optional **Kafka**: create modules and domain events but feature-flag them off by default (env var). Provide a dockerized Kafka locally.

2.1 Database schema (Postgres)

- `users(id, email, password_hash, role, created_at)`
- `patients(id, user_id, first_name, last_name, dob, phone, created_at)` (avoid storing address/PHI beyond minimal demo fields)
- `providers(id, user_id, specialty, timezone, created_at)`
- `availability(id, provider_id, start_time, end_time, status)`
- `appointments(id, patient_id, provider_id, start_time, end_time, status, telehealth_link, created_at)`
Indexes on foreign keys; unique constraints where appropriate.

2.2 Mongo collections

- `audit_events{ _id, actorId, action, targetType, targetId, timestamp, ip, userAgent }`
- `telehealth_sessions{ _id, appointmentId, roomSlug, createdAt, endedAt, notesNonPHI }`

2.3 APIs (examples)

- Auth: `/api/auth/register`, `/api/auth/login` → JWT
- Patient: CRUD profile (minimal), list my appointments
- Provider: CRUD availability, my schedule
- Appointment: search availability, book/reschedule/cancel; emits domain events (optional)
- Telehealth: `POST /api/telehealth/rooms` → `{ roomUrl }` (Jitsi link)
- Notification: `POST /api/notify/email` (demo)

3) Frontend

3.1 Staff Portal (React, Vite)

- Roles: **PROVIDER** and **ADMIN**.
- Pages: Login, Provider Dashboard (calendar view), Availability editor, Appointments list, Start session (launch Jitsi).
- UI: Minimal, sleek SaaS look; component library (e.g., Headless UI), Tailwind CSS, dark/light toggle.
- OpenAPI client generation for backend.
- E2E tests with **Playwright**.

3.2 Patient Portal (Vue 3, Vite)

- Pages: Login/Register, Book Appointment (provider + slot picker), My Appointments, Join Session (Jitsi link).
- Same design language; Vue Router + Pinia; OpenAPI client; Playwright e2e.

4) Local Dev & Testing

- **docker-compose.yml** runs: Postgres, Mongo, Zookeeper+Kafka (optional), all microservices, gateway, discovery, config, and both frontends (served via Node for local).
- **scripts/local_seed.sh** seeds demo users (patient/provider), availability slots, and a few appointments.
- **Makefile** with targets:
 - **make init** (mvn wrapper, node install, pre-commit hooks)
 - **make build** (backend mvn verify + frontend builds)
 - **make up / make down** (docker compose)
 - **make test** (unit + integration + e2e headless)
 - **make swagger** (openapi export)

- **Test strategy:**
 - Backend: JUnit 5, Testcontainers for Postgres/Mongo; Controller + Service + Repo tests; sample contract tests.
 - Frontend: Component tests + Playwright flows (login → book → join).
- Add a **Postman collection** (or `.http` files) for key flows.

5) Kubernetes (GKE) & Scaling

- K8s manifests for each service (Deployment, Service), namespace `pulsecare`.
- **Ingress** (NGINX) with TLS (self-signed for demo) + path-based routing to gateway.
- **HPA** for `appointment-service`: CPU target 60%, min 2 pods, max 8.
- `scripts/gke_deploy.sh` handles:
 1. Build & push images to **GCR** (or Artifact Registry).
 2. Create GKE cluster (standard or autopilot) if not exists.
 3. Apply manifests in correct order (secrets → config → db → discovery/config → services → gateway → ingress → hpa).
- `scripts/gke_destroy.sh` tears it down.
- `scripts/load_test_hpa.sh` uses **k6** or **hey** to hit appointment search/book APIs to trigger HPA scale-out; prints replica counts with `kubectl get hpa`.

6) CI/CD (GitHub Actions)

Workflow `.github/workflows/ci-cd.yaml`:

- Triggers on PR and `main`.
- Jobs:

- **Build & Test:** Maven verify (with unit + integration via Testcontainers); Node build + Playwright tests (headed in CI via xvfb).
- **Docker:** build & push to **GCR** using `$GCP_PROJECT_ID` and Workload Identity / SA key.
- **Deploy** (on tag): kubectl apply to GKE.
- Document required secrets:
 - `GCP_PROJECT_ID`, `GKE_CLUSTER`, `GKE_ZONE`, `GCP_SA_KEY` (JSON), `GCR_REPOSITORY`, `SENDGRID_API_KEY` (optional), `JITSI_BASE_URL` (default `https://meet.jit.si`).

7) Security & Compliance Notes

- JWT with short TTL and refresh strategy.
- No PHI/PII in logs; log masking util.
- CORS locked to frontends.
- HTTPS ingress; note that full HIPAA compliance requires additional controls (BAAs, key mgt, audit, backups). This demo is **HIPAA-friendly** but not compliance-certified.

8) Documentation

- **README.md** with:
 - 5-minute quickstart (local Docker)
 - Cloud deploy guide (GCR + GKE)
 - Architecture diagram (generated with Mermaid)
 - Service dependency graph
 - API reference links (OpenAPI UIs)

- “How AI assisted” section: prompt used, changes I made, review checklist
- Demo script (see below)

9) Demo Script (put into README)

1. Log in as provider; show availability editor and slots.
2. Log in as patient; book a slot; show appointment appears on provider’s dashboard.
3. Click **Start Telehealth** → opens Jitsi room; share link.
4. Run `scripts/load_test_hpa.sh` to simulate traffic and watch `appointment-service` scale from 2→N pods.
5. Open Grafana (optional) or actuator metrics to show health and latency.
6. Show GitHub Actions pipeline run and GCR images.

10) Automation & Quality Gates

- Pre-commit hooks (Spotless, Checkstyle, ESLint/Prettier).
- Multi-stage Dockerfiles with small runtimes (distroless or alpine).
- Health/readiness probes in K8s.
- Feature flags via environment variables.
- Seed users:
 - Patient: `patient1@demo.dev` / `Passw0rd!`
 - Provider: `provider1@demo.dev` / `Passw0rd!`
 - Admin: `admin@demo.dev` / `Passw0rd!`

11) Cloud Free-Tier Choices (document in README)

- **GCR/Artifact Registry + GKE** (use free credits).
- **DBs**: Use **Neon** (Postgres) and **MongoDB Atlas** free tiers; store connection strings in K8s Secrets.
- Note alternatives if free credits are exhausted.

12) Commands & Scripts (fill with working content)

- `scripts/gcp_login.sh`: gcloud auth, project set, artifact registry setup.
- `scripts/gke_deploy.sh`: build images, push, create cluster (if absent), apply manifests.
- `scripts/init_repo.sh`: creates remote GitHub repo and pushes code.
- `scripts/local_seed.sh`: inserts sample data via service APIs.
- `scripts/load_test_hpa.sh`: k6 (if installed) or **hey** load test; prints HPA metrics.

13) Acceptance Criteria

- `make up` starts everything locally; both SPAs reachable behind gateway.
- Booking flow works end-to-end; Jitsi link is generated.
- `make test` runs and passes (backend + frontend + e2e).
- `gke_deploy.sh` deploys to GKE; public ingress URL works.
- HPA scales `appointment-service` under load.
- README has architecture diagram, demo script, AI-assist notes.
- GitHub Actions passes on PR and deploys on tag.

14) After Code Generation

- Print a short checklist of env vars and credentials needed (GitHub, GCP, Neon, MongoDB Atlas, SendGrid).
- Print the `kubectl get ingress -n pulsecare` command to retrieve the public URL after deploy.

Now generate the entire repository with working code, tests, Dockerfiles, K8s manifests, scripts, and CI/CD. Do not stub—implement minimal working features. Ask for any missing credentials at run time and produce sample `.env.example` files.

How to use (you)

1. Paste the prompt above into **Cursor** at the repo root (or a fresh folder).
2. Let Cursor scaffold. When asked for tokens/IDs, use placeholders; you'll fill `.env` later.
3. Run locally: `make up` → open the URLs from README.
4. Run tests: `make test`.
5. Initialize GitHub: `bash scripts/init_repo.sh`.
6. Deploy to cloud:
 - `bash scripts/gcp_login.sh`
 - `bash scripts/gke_deploy.sh`
 - Get ingress URL: `kubectl get ingress -n pulsecare`

Efficiency Metrics (for your interview)

- **Scope (mid-complexity):** 9 Spring microservices, 2 SPAs, Docker, K8s, CI/CD, HPA, hybrid DBs, auth & gateway, tests.

- **Est. baseline without AI:** ~2 dev-weeks (60–80 hours) for full scaffolding + wiring + docs + basic tests.
 - **With Cursor (you as lead + AI as “junior”):** ~4–6 hours hands-on to review prompts, tweak configs, connect secrets, run deploys.
 - **Time saved:** ~56–74 hours ($\approx$ 85–92% reduction).
 - **Throughput uplift:** You acting as architect/reviewer achieves ~10–15x bootstrapping speed vs. manual greenfield setup.
 - **Quality gates:** Static analysis, Testcontainers integration tests, Playwright e2e, CI/CD — comparable to a small team’s first sprint Definition of Done.
-

Pitfalls to watch (and how to handle during the demo)

- **GKE quotas / free credits:** Have a fallback to local `docker-compose` demo; keep `kubectl` and screenshots ready.
 - **DB connectivity from GKE:** Prefer public-access free tiers (Neon, Atlas) with IP allowlists.
 - **Ingress DNS/TLS delay:** Provide self-signed TLS option; demo locally if DNS lags.
 - **Playwright dependencies in CI:** Ensure the workflow installs Playwright browsers before tests.
 - **JWT clock skew:** Set small skew tolerance.
-

If you want, I can also generate a **1-page interviewer handout** (architecture diagram + bullet demo steps) and a **scripted talk track** tailored to this project.